

WEEK 3 - INTERNSHIP ASSIGNMENT

TOPIC: Superstore SQL Analysis

Step 1: Setup Data –

Import the Superstore dataset into a table named `superstore_raw`.

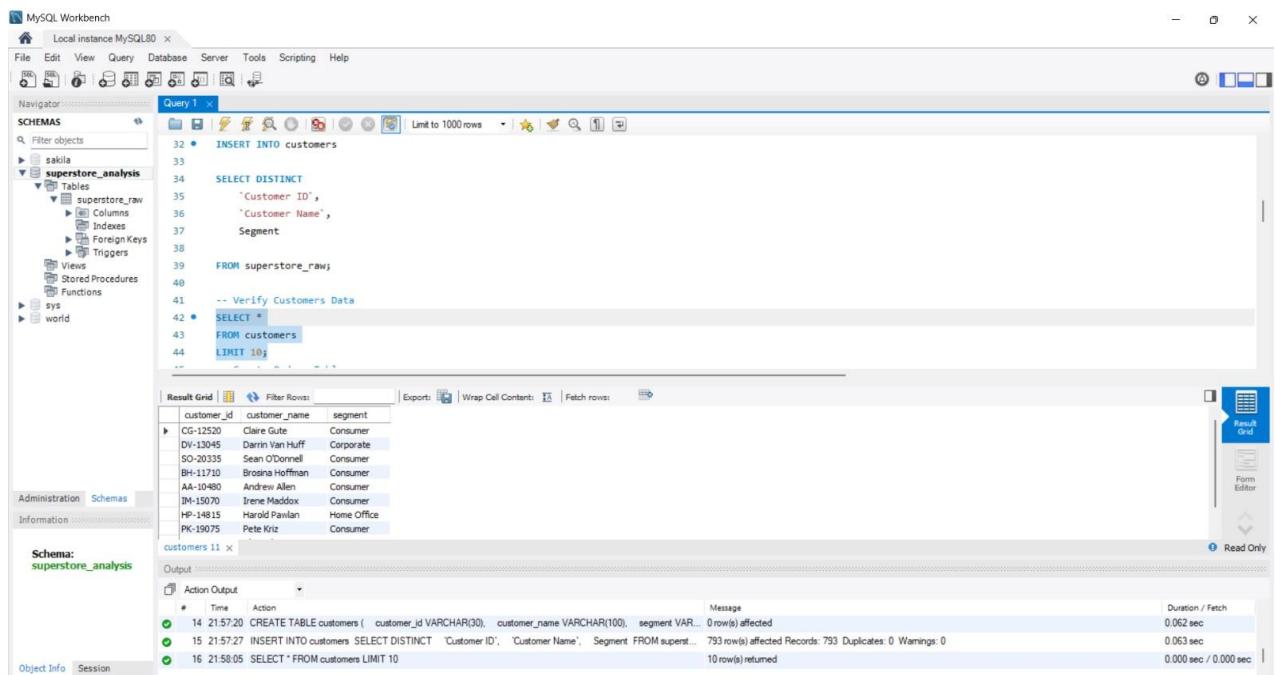
Create these 3 tables from it:

`customers`

`orders`

`products`

Insert data into these tables using `SELECT DISTINCT`.



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' panel with a tree view containing 'sakila' and 'superstore_analysis'. The 'superstore_analysis' schema is selected, showing a table 'superstore_raw'. The main query editor shows the following SQL code:

```
32 INSERT INTO customers
33
34 SELECT DISTINCT
35     'Customer ID',
36     'Customer Name',
37     Segment
38
39 FROM superstore_raw;
40
41 -- Verify Customers Data
42 *
43 FROM customers
44 LIMIT 10;
```

The 'Result Grid' shows the first 10 rows of the 'customers' table:

customer_id	customer_name	segment
CG-12520	Claire Gule	Consumer
DV-13045	Darin Van Huff	Corporate
SO-20335	Sean O'Donnell	Consumer
BH-11710	Brosina Hoffman	Consumer
AA-10480	Andrew Allen	Consumer
JM-15070	Irene Maddox	Consumer
HP-14815	Harold Pavilan	Home Office
PK-19075	Pete Kriz	Consumer

The 'Action Output' panel at the bottom shows the execution of the SQL commands:

#	Time	Action	Message	Duration / Fetch
14	21:57:20	CREATE TABLE customers (customer_id VARCHAR(30), customer_name VARCHAR(100), segment VAR...	0 row(s) affected	0.062 sec
15	21:57:27	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment FROM super...	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.063 sec
16	21:58:05	SELECT * FROM customers LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

superstore_analysis

Tables

superstore_raw

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

world

Query 1

```

72      'Customer ID',
73      Sales,
74      Quantity,
75      Discount,
76      Profit
77
78 FROM superstore_raw;
79
80 -- Verify Orders Data
81
82 SELECT *
83 FROM orders
84 LIMIT 10;

```

Result Grid

row_id	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CS-12520	261.96	2	0	41.9136
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CS-12520	731.94	3	0	219.582
3	CA-2016-136688	6/12/2016	6/16/2016	Second Class	DH-13045	14.62	2	0	6.8714
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	22.368	2	0.2	2.5164
6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	48.86	7	0	14.1694
7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	7.28	4	0	1.9658
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	907.152	6	0.2	90.7152

orders 12 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
19	22:01:20	CREATE TABLE orders (row_id INT, order_id VARCHAR(50), order_date VARCHAR(50), ship_date VA...	0 row(s) affected	0.032 sec
20	22:01:56	INSERT INTO orders SELECT DISTINCT 'Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mod...	9694 row(s) affected Records: 9694 Duplicates: 0 Warnings: 0	0.203 sec
21	22:02:30	SELECT * FROM orders LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

superstore_analysis

Tables

superstore_raw

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

world

Query 1

```

101      'Product ID',
102      Category,
103      'Sub-Category',
104      'Product Name'
105
106 FROM superstore_raw;
107
108 -- Verify Products Data
109
110 SELECT *
111 FROM products
112 LIMIT 10;
113

```

Result Grid

product_id	category	sub_category	product_name
FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase
FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...
OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...
FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table
OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N' Roll Cart System
FUR-FU-10001487	Furniture	Furnishings	Eldon Expressions Wood and Plastic Desk Acces...
OFF-AH-10002833	Office Supplies	Art	Newell 322
TEC-PH-10002275	Technology	Phones	Mitel 5320 IP Phone VoIP phone

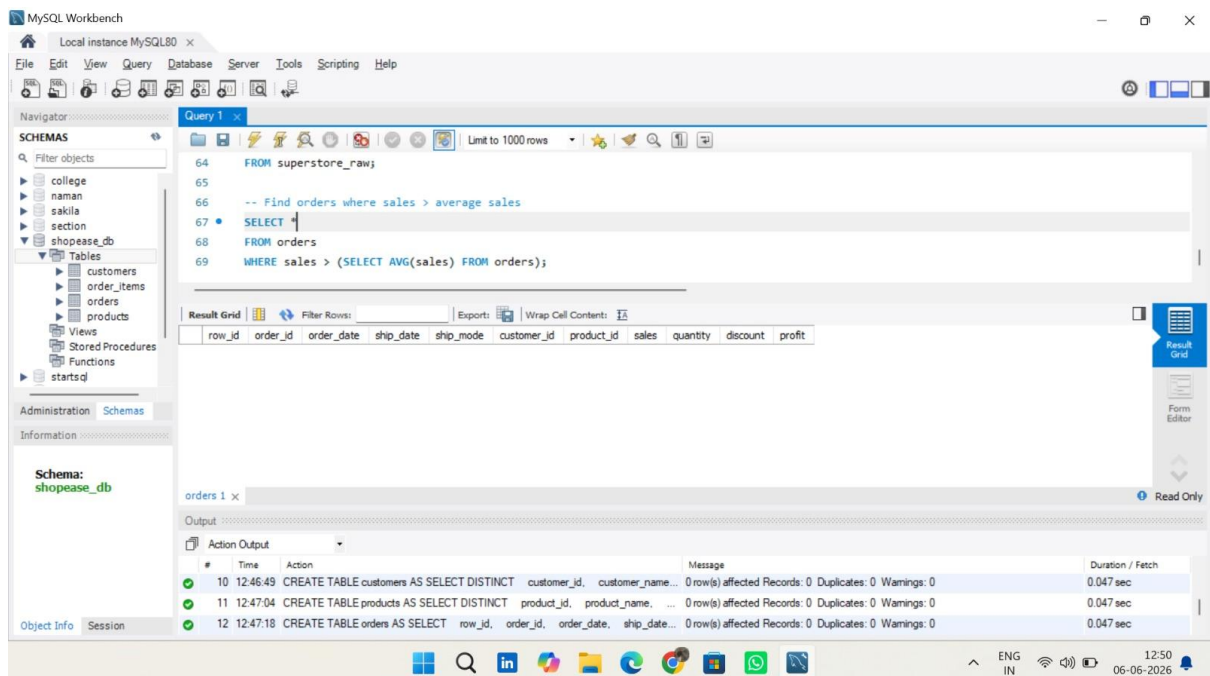
products 13 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
24	22:06:27	CREATE TABLE products (product_id VARCHAR(50), category VARCHAR(50), sub_category VARCHAR...	0 row(s) affected	0.266 sec
25	22:06:33	INSERT INTO products SELECT DISTINCT 'Product ID', Category, 'Sub-Category', 'Product Name' F...	1842 row(s) affected Records: 1842 Duplicates: 0 Warnings: 0	0.313 sec
26	22:07:03	SELECT * FROM products LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Object Info Session



Query 2: Highest Sales Order Per Customer -

Using Correlated Subquery –

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```
105 JOIN (
106     SELECT customer_id, MAX(sales) AS max_sales
107     FROM orders
108     GROUP BY customer_id
109 )
```

The result grid displays the following data:

row_id	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.184	9	0.2	85.3092
25	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.2649
28	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.0522
36	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.4737
39	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764
55	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.6855
58	CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	319.41	5	0.1	7.098
68	CA-2014-106376	17/5/2014	17/10/2014	Standard Class	RS-11590	1113.024	8	0.2	111.3024

The output pane shows the following messages:

```
46 23:01:52 SELECT * FROM orders o WHERE sales = ( SELECT MAX(sales) FROM orders ... Error Code: 2013. Lost connection to MySQL server during query 30.000 sec
47 23:03:28 SELECT * FROM orders WHERE sales > ( SELECT AVG(sales) FROM orders ) LIMIT 1000 row(s) returned 0.016 sec / 0
48 23:05:54 SELECT o.* FROM orders o JOIN ( SELECT customer_id, MAX(sales) AS max_sales... 795 row(s) returned 0.015 sec / 0
```

Query 3: Total Sales Per Customer -

Using CTE –

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query is as follows:

```
113
114 -- Calculate total sales for each customer
115 WITH customer_sales AS (
116     SELECT
117         customer_id,
118         SUM(sales) AS total_sales
119     FROM orders
120     GROUP BY customer_id
121 )
122 SELECT * FROM customer_sales;
```

The result grid displays the following data:

customer_id	total_sales
CG-12520	1148.7800000000002
DV-13045	1119.4830000000002
SO-20335	2602.5750000000001
BH-11710	6255.3510000000001
AA-10480	1782.872
JM-15070	4930.4739999999999

The output pane shows the following message:

```
47 23:03:28 SELECT * FROM orders WHERE sales > ( SELECT AVG(sales) FROM orders ) LIMIT 1000 row(s) returned 0.016 sec / 0.000 sec
```

Query 4: Customers Above Average Sales –

Using CTE + Subquery –

The screenshot shows the MySQL Workbench interface with a SQL query in the editor. The query uses a Common Table Expression (CTE) to calculate total sales for each customer and then filters for customers whose total sales are greater than the average total sales.

```
124 -- Customers whose total sales > average customer sales
125 WITH customer_sales AS (
126     SELECT
127         customer_id,
128         SUM(sales) AS total_sales
129     FROM orders
130     GROUP BY customer_id
131 )
132 SELECT *
133 FROM customer_sales
134 WHERE total_sales > (
135     SELECT AVG(total_sales) FROM customer_sales
136 );
```

The result grid shows the following data:

customer_id	total_sales
BH-11710	6255.351000000001
IM-15070	4930.4739999999999
PK-19075	8158.654
TB-21520	4730.628
MA-17560	4280.6209999999999

Query 5: Customer Ranking -

Using RANK() –

The screenshot shows the MySQL Workbench interface with a SQL query in the editor. The query uses the RANK() function to rank customers based on their total sales.

```
133 FROM customer_sales
134 WHERE total_sales > (
135     SELECT AVG(total_sales) FROM customer_sales
136 );
137
138 -- Rank customers based on total sales
139 SELECT
140     customer_id,
141     SUM(sales) AS total_sales,
142     RANK() OVER (ORDER BY SUM(sales) DESC) AS rank_position
143 FROM orders
144 GROUP BY customer_id;
```

The result grid shows the following data:

customer_id	total_sales	rank_position
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5
SC-20095	14142.333999999999	6
KL-16645	14071.917	7

Query 6: Row Number Per Customer –

Using ROW_NUMBER() –

SCHEMAS

- Filter objects
- college
- naman
- sakila
- section
- starts
- superstore_db
 - superstore_raw
 - Views
 - Stored Procedures
 - Functions
- sys
- world
- zerotrust

Administration | **Schemas**

Information

Schema: **superstore_db**

```

144 GROUP BY customer_id;
145
146 -- Assign row numbers to orders within each customer
147 • SELECT
148     customer_id,
149     order_id,
150     sales,
151     ROW_NUMBER() OVER (
152         PARTITION BY customer_id
153         ORDER BY sales DESC
154     ) AS row_num
155 FROM orders;
  
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	order_id	sales	row_num
AA-10315	CA-2016-103982	3930.072	1
AA-10315	CA-2014-128055	673.568	2
AA-10315	CA-2016-103982	431.976	3
AA-10315	CA-2017-147039	362.94	4
AA-10315	CA-2014-128055	52.98	5
AA-10315	CA-2016-103982	41.72	6
AA-10315	CA-2015-121391	26.96	7

Result 16 x | Read Only

Output

Action Output

Query 7: Top 3 Customers -

Using Window Function -

Navigator

SCHEMAS

- Filter objects
- college
- naman
- sakila
- section
- starts
- superstore_db
 - superstore_raw
 - Views
 - Stored Procedures
 - Functions
- sys
- world
- zerotrust

Administration | **Schemas**

Information

Schema: **superstore_db**

```

156
157 -- Top 3 customers based on total sales
158 • SELECT *
159 FROM (
160     SELECT
161         customer_id,
162         SUM(sales) AS total_sales,
163         RANK() OVER (ORDER BY SUM(sales) DESC) AS rnk
164     FROM orders
165     GROUP BY customer_id
166 ) t
167 WHERE rnk <= 3;
  
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	total_sales	rnk
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3

Result 17 x | Read Only

Output

Action Output

Final Combined Query -

JOIN + CTE + Window Function -

Navigation

SCHEMAS

Filter objects

college

naman

sakila

section

startsdl

superstore_db

Tables

superstore_raw

Views

Stored Procedures

Functions

sys

world

zerotrust

Administration

Schemas

Limit to 1000 rows

168

169

170

171

172

173

174

175

176

177

178

179

180

181

-- Final query combining JOIN + CTE + Window Function

WITH customer_sales AS (

SELECT

customer_id,

SUM(sales) AS total_sales

FROM orders

GROUP BY customer_id

)

SELECT

c.customer_name,

cs.total_sales

Information

Schema: superstore_db

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Result Grid

customer_name	total_sales	rank_position
Sean Miller	25043.05	1
Tamara Chand	19017.847999999998	2
Raymond Buch	15117.339	3
Tom Ashbrook	14595.62	4
Adrian Barton	14355.610999999997	5

Result 18 x

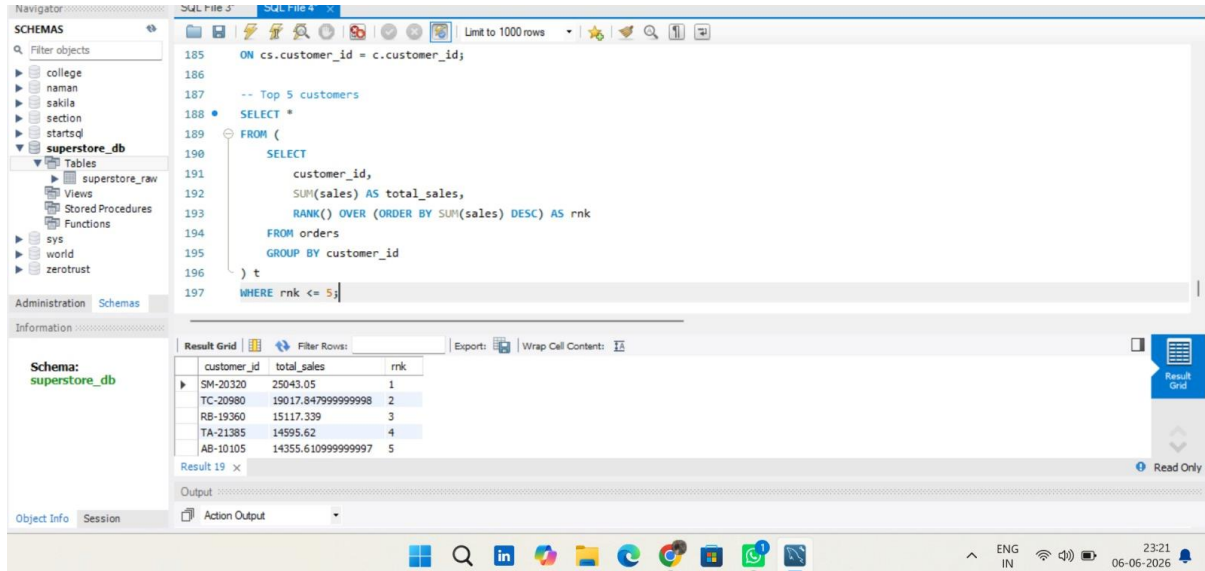
Read Only

Output

Action Output

Mini Project: Customer Sales Insights –

Top 5 Customers –



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'superstore_db' schema with tables like 'superstore_raw', 'views', 'stored_procedures', and 'functions'. The central pane contains a SQL query to identify the top 5 customers based on total sales. The query uses a subquery to calculate the total sales for each customer, ranked in descending order, and then filters for the top 5 customers.

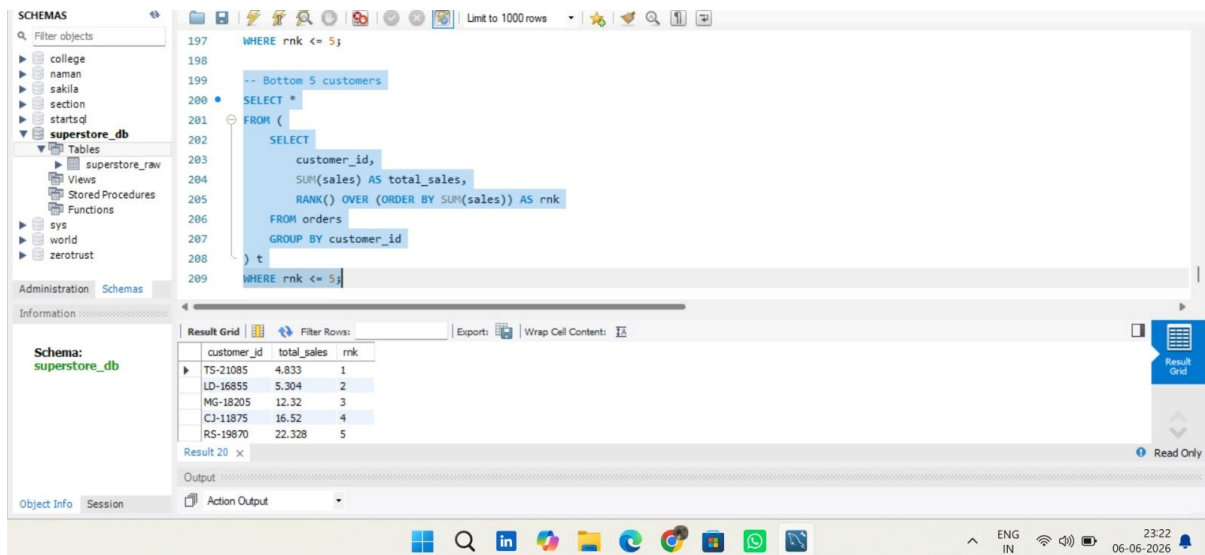
```
185 ON cs.customer_id = c.customer_id;
186
187 -- Top 5 customers
188 SELECT *
189 FROM (
190     SELECT
191         customer_id,
192         SUM(sales) AS total_sales,
193         RANK() OVER (ORDER BY SUM(sales) DESC) AS rnk
194     FROM orders
195     GROUP BY customer_id
196 ) t
197 WHERE rnk <= 5;
```

The right pane shows the 'Result Grid' with the following data:

customer_id	total_sales	rnk
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3
TA-21385	14595.62	4
AB-10105	14355.610999999997	5

Bottom 5 Customers –

Customers Who Made Only One Order –



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'superstore_db' schema. The central pane contains a SQL query to identify the bottom 5 customers based on total sales. The query uses a subquery to calculate the total sales for each customer, ranked in ascending order, and then filters for the bottom 5 customers.

```
197 WHERE rnk <= 5;
198
199 -- Bottom 5 customers
200 SELECT *
201 FROM (
202     SELECT
203         customer_id,
204         SUM(sales) AS total_sales,
205         RANK() OVER (ORDER BY SUM(sales)) AS rnk
206     FROM orders
207     GROUP BY customer_id
208 ) t
209 WHERE rnk <= 5;
```

The right pane shows the 'Result Grid' with the following data:

customer_id	total_sales	rnk
TS-21085	4.833	1
LD-16855	5.304	2
MG-18205	12.32	3
CJ-11875	16.52	4
RS-19870	22.328	5

Above Average Customers –

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with 'superstore_db' selected. The central pane shows a SQL query in 'SQL File 4':

```
-- Customers with above average total sales
205
206 -- Execute the selected portion of the script or everything, if there is no selection
207
208 SELECT
209     customer_id,
210     SUM(sales) AS total_sales
211 FROM orders
212 GROUP BY customer_id
213
214 SELECT *
215 FROM customer_sales
216 WHERE total_sales > (
217     SELECT AVG(total_sales) FROM customer_sales
218 );
```

The right pane shows the 'Result Grid' with the following data:

customer_id	total_sales
BH-11710	6255.351000000001
IM-15070	4930.473999999999
PK-19075	8158.654
TB-21520	4730.628
MA-17560	4280.620999999999

The bottom status bar shows the system time as 23:24 on 06-06-2026.

Highest Order Value Per Customer –

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with 'superstore_db' selected. The central pane shows a SQL query in 'SQL File 4':

```
214 FROM customer_sales
215 WHERE total_sales > (
216     SELECT AVG(total_sales) FROM customer_sales
217 );
218
219 -- Highest order value per customer
220 SELECT *
221 FROM orders o
222 WHERE sales = (
223     SELECT MAX(sales)
224     FROM orders
225     WHERE customer_id = o.customer_id
226 );
```

The right pane shows the 'Result Grid' with the following data:

customer_id	total_sales
BH-11710	6255.351000000001
IM-15070	4930.473999999999
PK-19075	8158.654
TB-21520	4730.628
MA-17560	4280.620999999999

The bottom status bar shows the system time as 23:28 on 06-06-2026.

